
Human vs. AI-Generated Text: From Linguistic Analysis to Zero-Shot Detection Methods

Louis-Armand Aumont
ENSAE Paris
louis-armand.aumont@ensae.fr

Abstract

Detecting LLMs-generated texts has become a challenging task with the exponential improvement of LLMs. Following a corpus analysis we extract some stylistic and stylometric characteristics of LLMs-generated texts using Yule’s K measure and POS tagging. We later implement zero-shot methods in the white-box setting to classify these texts between LLM and Human generated. We focus on zero-shot methods as they do not require knowing which LLM model generated the text and do not require heavy compute which is the more realistic setup. On the test set, the best performing model in terms of AUROC is Binoculars with an AUROC of 0.8745.

1 Introduction and Problem Framework

With the release of GPT-2 in February 2019 and its ability to generate fluid and coherent text, the prevalence of LLM-generated content has grown dramatically. As computational resources and generation techniques continue to be improved, it has become challenging to detect an LLM-generated text. Detecting such texts is vital as they can convey biases, fake news or report purely invented facts. Over the years, Machine-Learning techniques ranging from supervised to unsupervised ones have been developed to detect those LLM texts. The methods exposed in this article will focus on detecting AI models that have not been explicitly designed to be identifiable, contrary to watermarked ones. Thus, zero-shot methods leverage intrinsic properties of AI-generated texts such as the log-probability function curvature.

The problem can be framed as an unsupervised classification task. The goal is to assess if a given text x , is sampled from a source model p_θ and thus knowing if it is AI-generated. The study is carried out in the white box setting, a setting where we do not suppose access to model’s architecture or parameters. This setting is the more realistic one as all the LLMs providers do not grant access to their models parameters.

The code used to produce the entire analysis is publicly available.¹

2 Corpus Analysis

Lexical Diversity Analysis

We analyze the dataset composed of human and AI-generated texts aiming to extract the particularities of AI-generated texts as opposed to human written ones.

The original Kaggle dataset² is composed of around 500,000 essays aggregated from different sources. The original dataset is highly imbalanced with 305,797 (62.76%) human written texts and only 181,434 (37.24%) AI-generated ones. The dataset is cleaned from misannotation, duplicates and empty texts. To properly reflect statistical properties of both classes, we sample texts from the Human class in order to construct a balanced dataset, $\mathcal{D}$, of 362,868 texts.

One of the very basic Natural Language Processing (NLP) techniques is to compare word counts between AI and Human generated texts as well as the vocabulary richness. To compute those we assume that each word is a token and vocabulary richness is defined as the number of unique token in each text.

¹https://github.com/LouisArmand0/Detect_LLMs_generated_texts

²<https://www.kaggle.com/datasets/shanegerami/ai-vs-human-text?resource=download>

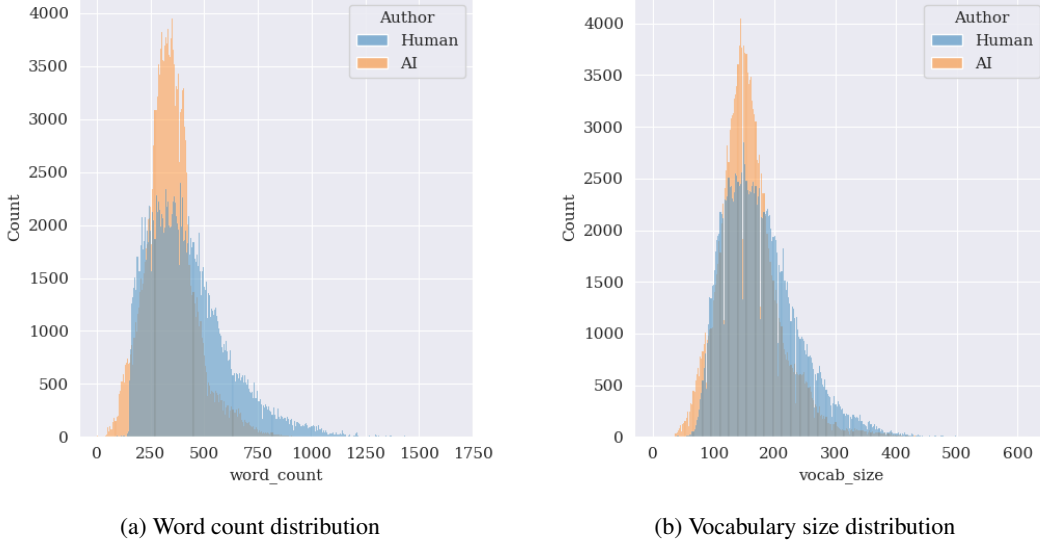


Figure 1: Distribution of features by author type (Human vs AI)

Fig. 1a shows that AI-generated texts tend to have a length between 250 and 500 words while the Human texts length is more spread out. Fig. 1b exhibits that Human texts tend to have more vocabulary richness as shown by the right fat tail of the human text distribution.

This measure of vocabulary richness is too simple as, in our corpus, human texts tend to be longer, which could artificially inflate their vocabulary size. We therefore compute two length-robust measures: the Moving Average Type-Token Ratio (MATTR) Covington and McFall [2010] and Yule’s K , which allow us to assess whether AI and human-generated texts exhibit statistically significant differences in vocabulary richness.

The MATTR is a length-corrected Type-Token Ratio and serves as a local measure of lexical diversity computed over a sliding window:

$$MATTR = \frac{1}{N - w + 1} \sum_{i=1}^{N-w+1} \frac{|\mathcal{V}(w_i)|}{w}, \quad (1)$$

where N is the total number of tokens, w is the window size (here, $w = 100$), and $|\mathcal{V}(w_i)|$ is the number of unique types in the window starting at position i .

Complementary to MATTR, Yule’s K provides a global measure of vocabulary richness based on the frequency distribution of words across the entire text:

$$K = 10^4 \cdot \frac{\sum_{i=1}^V f_i \cdot i^2 - N}{N^2}, \quad (2)$$

where N is the total number of tokens, V is the vocabulary size, and f_i is the number of words appearing exactly i times. A higher K indicates greater repetitiveness, while a lower K reflects richer vocabulary.

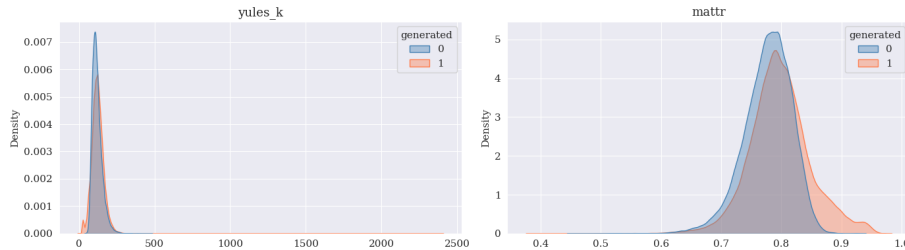


Figure 2: MATTR and Yule’s K distributions per class.

As shown in Fig. 2, the two distributions overlap substantially. To quantify the differences, we report the class means alongside Cohen’s d as a measure of effect size, and conduct a Mann-Whitney U test, a non-parametric test suited for non-normally distributed data, to assess statistical significance.

Both features are statistically significant at the 1% level. However, effect sizes reveal an important nuance: while Yule’s K exhibits a small effect ($d = 0.10$), MATTR shows a medium effect ($d = 0.44$), suggesting

Feature	Human (mean)	AI (mean)	Cohen’s d	p-value
MATTR	0.777	0.797	0.438	< 0.01
Yule’s K	119.50	123.19	0.102	< 0.01

Table 1: Comparison of lexical diversity features between Human and AI-generated texts with effect sizes and Mann-Whitney U test results.

that local lexical diversity is a more reliable discriminator between Human and AI-generated texts than global repetitiveness.

Interestingly, AI texts score higher on MATTR, indicating greater local lexical diversity, a counterintuitive result suggesting that modern LLMs are well-calibrated to avoid local word repetition. However, AI texts also score higher on Yule’s K , revealing greater global repetitiveness: while LLMs vary their vocabulary locally, they tend to rely on the same core vocabulary throughout the entire text.

Part-Of-Speech Tagging

To further characterize the stylometric differences between AI and Human generated texts, we analyze the Part-of-Speech (POS) distribution of each class. POS tags describe the grammatical role of each word (noun, verb, adjective, etc.) and reflect the syntactic structure of a text. POS tagging is performed using the spaCy *en_core_web_sm* model, chosen for its computational efficiency on a large corpus of 362,868 texts, as the larger transformer-based models (*en_core_web_trf*) would incur high compute costs at this scale Honnibal et al. [2020].

POS analysis reveals stylistic differences between Human and AI generated texts. LLMs tend to use a more nominal and descriptive writing style, shown by a higher proportion of nouns and adjectives in its texts (Fig. 3a). It is consistent with the global repetitiveness exhibited by the Yule’s K measure. On the contrary, Human written texts from the corpus tend to exhibit a higher proportion of auxiliary verbs and adverbs, suggesting a more conversational tone. We complement this first POS tagging by computing POS bigrams on the corpus to uncover finer syntactic differences between AI and Human generated texts. Indeed, POS tagging and counting only capture the overall composition of the text and do not account for the *sequential structure* of the text. To capture syntactic patterns, we extend the analysis to POS bigrams, defined as consecutive pairs of POS tags (t_i, t_{i+1}) . The proportion of each bigram b in a text of N tokens is calculated as:

$$p(b) = \frac{\text{count}(t_i \rightarrow t_{i+1})}{N - 1} \quad (3)$$

This allows us to detect recurring syntactic constructions, such as ADJ→NOUN for noun phrase elaboration or AUX→VERB for verbal constructions, which are not revealed by a unigram level analysis (a simple POS tagging). As shown in Fig. 3b, the most discriminative POS bigram for AI is NOUN→PUNCT, suggesting that LLMs tend to end noun phrases with punctuation, revealing a more declarative and assertive style. Moreover, with the high frequency of ADJ→NOUN bigrams, it confirms that LLMs use an assertive style with rich noun phrase elaboration. On the other hand, the most discriminative bigram for human texts is DET→NOUN, suggesting more natural and grounded nominal references, combined with greater use of auxiliary constructions (AUX→PART, AUX→VERB), consistent with a more conversational style.

The lexical and syntactic features identified in this corpus analysis provide a characterization of the stylometric differences between Human and AI-generated texts, motivating the zero-shot detection methods developed in Section 3.

3 Detection Algorithms

We implement several state-of-art zero-shot detection models, namely DetectGPT and Binoculars. By definition these models do not need to be trained on a proper training set, they only require to calibrate the classification threshold on a validation set. In a production setting, we can think of adjusting the threshold in a dynamic manner depending on the texts analyzed so far. Like in online learning theory, the updated threshold should converge to the best threshold assuming a large sample of texts.

DetectGPT

DetectGPT, Mitchell et al. [2023], emanates from a key observation of the log-probability function curvature of large language models, as shown in Fig. 4. LLMs-generated texts tend to lie in a region of local maxima of the model’s log-probability function, while human-written texts do not. The intuition is that LLMs generates

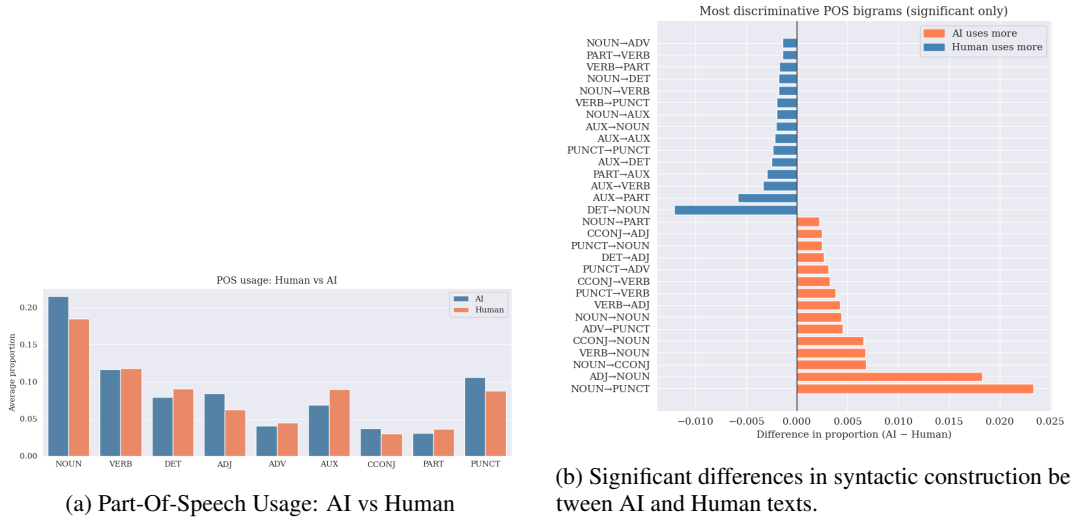
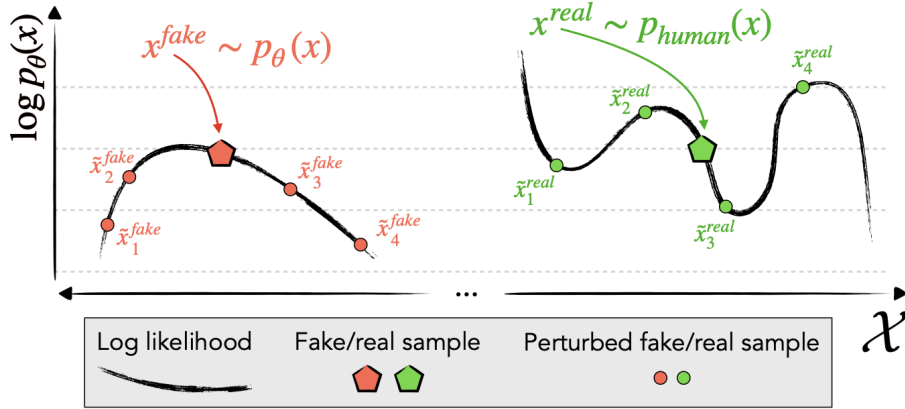


Figure 3: POS unigram and bigram analysis of the corpus

text by repeatedly selecting high-probability tokens, thus producing sequences that lie in a local maxima of the log-probability function. By contrast, Humans do not optimize any loss function while writing texts, thus producing a more varied text.



We denote by $\log p_\theta(x)$ the log-probability of a text x under a language model p_θ . DetectGPT algorithm relies on the hypothesis that if a text x is AI-generated, then a small perturbation $\tilde{x}$ of x will on average have a lower log-probability than itself, since x is assumed to live near a local maxima of the log-probability function. On the contrary, if x is human-written perturbations $\tilde{x}$ will not systematically differ from x in log-probability, as human text does not live near local maxima.

This intuition is formalized through the following score:

$$d(x, p_\theta, q) = \log p_\theta(x) - \mathbb{E}_{\tilde{x} \sim q(\cdot|x)} [\log p_\theta(\tilde{x})], \quad (4)$$

where $q(\cdot|x)$ is a perturbation distribution that generates semantically similar variants of x by masking and infilling spans of text. A high score $d(x, p_\theta, q)$ indicates that x lies at a local maximum of $\log p_\theta$, suggesting machine generation. In practice, the expectation is approximated by Monte Carlo sampling:

$$d(x, p_\theta, q) \approx \log p_\theta(x) - \frac{1}{k} \sum_{i=1}^k \log p_\theta(\tilde{x}_i), \quad \tilde{x}_i \sim q(\cdot|x), \quad (5)$$

where k is the number of perturbations (here $k = 10$).

The perturbation distribution $q(\cdot|x)$ is implemented using a masked language model. Following the original paper, we randomly mask a fraction $\gamma = 0.15$ of tokens in x , replacing them with sentinel tokens, and infill the masked spans using T5 Raffel et al. [2023].

As mentioned in Sec. 1, DetectGPT algorithm operates in the white-box setting, it only requires the ability to compute $\log p_\theta(x)$ for any text x , without access to the source model’s architecture or parameters. The scoring model p_θ is therefore a surrogate model, here GPT-2, used as a proxy for the unknown source model. We can now properly write the algorithm pseudo-code given as follows:

Algorithm 1 DetectGPT Algorithm

Require: Text x , scoring model p_θ , perturbation model q , number of perturbations k , threshold η

- 1: Compute $\log p_\theta(x)$
- 2: **for** $i = 1$ to k **do**
- 3: Sample $\tilde{x}_i \sim q(\cdot|x)$ by masking and infilling spans of x
- 4: Compute $\log p_\theta(\tilde{x}_i)$
- 5: **end for**
- 6: $d \leftarrow \log p_\theta(x) - \frac{1}{k} \sum_{i=1}^k \log p_\theta(\tilde{x}_i)$
- 7: **return** $\mathbf{1}[d > \eta]$ $\triangleright 1 = \text{AI-generated}$

Binoculars Algorithm

The Binoculars algorithm, Hans et al. [2024], relies on comparing how two different language models evaluate the same sample text x . Instead of using a single model’s likelihood, like DetectGPT, it measures the discrepancy between two models. The intuition is that LLM-generated text is often better aligned with the distribution of certain models, which creates detectable differences when evaluated by another model. Said differently, the intuition is that LLM-generated text is typically optimized to be highly probable under the generating model (as mentioned in the DetectGPT algorithm). The reference model p_{θ_r} may assign a relatively low negative log-likelihood, i.e., a high probability, to such text. In contrast, the observer model p_{θ_q} may assign a higher negative log-likelihood:

$$\ell_q(x) > \ell_r(x) \quad .$$

For human-written text, the likelihood estimates tend to be more consistent across models:

$$\ell_q(x) \approx \ell_r(x).$$

This key finding is crystallized in the following scoring mechanism. Let $x = (x_1, \dots, x_T)$ be a sequence of tokens. Consider two pretrained language models: a reference model p_{θ_r} and an observer model p_{θ_q} .

For each model, we compute the average negative log-likelihood for a given model p_{θ_k} :

$$\ell_k(x) = -\frac{1}{T} \sum_{t=1}^T \log P_{p_{\theta_k}}(x_t \mid x_{<t}), \quad (6)$$

The Binoculars score is then defined as:

$$S(x) = \ell_q(x) - \ell_r(x). \quad (7)$$

Algorithm 2 Binoculars Zero-Shot Detection

Require: Text $x = (x_1, \dots, x_T)$, reference model M_r , observer model M_o , threshold τ

- 1: Compute $\ell_r(x) \leftarrow -\frac{1}{T} \sum_{t=1}^T \log P_{M_r}(x_t \mid x_{<t})$
- 2: Compute $\ell_o(x) \leftarrow -\frac{1}{T} \sum_{t=1}^T \log P_{M_o}(x_t \mid x_{<t})$
- 3: $S(x) \leftarrow \ell_o(x) - \ell_r(x)$
- 4: **return** $\mathbf{1}[S(x) > \tau]$ $\triangleright 1 = \text{AI-generated}$

The Binocular algorithm also works in the white box setting, in which we are working as defined in Sec. 1, as it only requires to compute the log-probabilities without needing access to the model’s architecture or parameter.

4 Results

Because of computation power limitation, we sample 200 texts from the corpus with an equal number of AI and Human generated texts. As no training dataset is needed, we split the sample dataset in two, a validation set and a test set. The validation set is used to calibrate the classification thresholds of the different models.

Defining a competitive baseline

Before assessing the performance of the two methods previously exposed, we first define some benchmark in order of increasing complexity.

The first benchmark is simple, it always predicts one. As the dataset is balanced it is right half of the time.

The second benchmark leverages the core idea of DetectGPT algorithm, it computes the log-probabilities of the sample texts, scale them between 0 and 1 and following an optimized threshold it classifies the text x as 1 if AI-generated and 0 if Human-written.

The third benchmark is the perplexity of a text. Given a text $x = (x_1, \dots, x_T)$ and a scoring model p_θ , the average negative log-likelihood is

$$-\frac{1}{T} \sum_{t=1}^T \log p_\theta(x_t | x_{<t}),$$

and perplexity is defined as its exponential:

$$\text{PPL}(x) = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log p_\theta(x_t | x_{<t}) \right). \quad (8)$$

This normalization makes the score independent of text length and allows fair comparison across samples, with lower perplexity indicating texts that are more probable under the model.

Each of the threshold is calibrated using the validation set by maximizing the $F1$ score. The $F1$ score is defined as:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (9)$$

Quantitative results

As reported in Fig. 5 and Table 2, perplexity alone is not sufficient to reliably discriminate between Human and AI-generated texts, as it yields only moderate performance (AUROC = 0.7011).

In contrast, methods leveraging multiple signals significantly improve detection performance. In particular, Binoculars, which compares log-likelihoods across two different language models, achieves the best performance with an AUROC of 0.8745, highlighting the benefit of model disagreement as a detection signal.

DetectGPT, which relies on local perturbations of the input text to estimate the curvature of the log-probability function, also improves performance over raw likelihood methods, reaching an AUROC of 0.7517. However, this gain comes at a significant computational cost, as the method requires multiple forward passes over perturbed samples, making it substantially slower than Binoculars.

Finally, using the raw log-probability of a single model already provides a reasonable baseline with an AUROC of 0.7480, confirming that likelihood-based signals contain meaningful information for detection, although they are suboptimal when used in isolation.

Model	Accuracy	AUROC	Runtime (200 texts)
Log Probability	0.7350	0.7480	seconds
Perplexity	0.6150	0.7011	seconds
DetectGPT	0.7350	0.7517	25min 17s
Binoculars	0.7300	0.8745	3min 21s

Table 2: Performance comparison of zero-shot AI text detection methods. Runtime is measured per text instance (in seconds).

As shown in Fig. 6, all methods perform above random chance, but with significant differences in ranking quality. Binoculars clearly dominates the other approaches, indicating that cross-model disagreement provides a stronger detection signal than either raw likelihood or perturbation-based scoring. DetectGPT and log-probability perform similarly, suggesting that perturbation-based curvature provides only a marginal gain over single-model likelihood. Perplexity yields the weakest performance among the baselines. Moreover, the main distinction between methods is not only the absolute AUC values, but also the steepness of the ROC curve at low false-positive rates, which is critical in practical detection settings where false positives must be minimized.

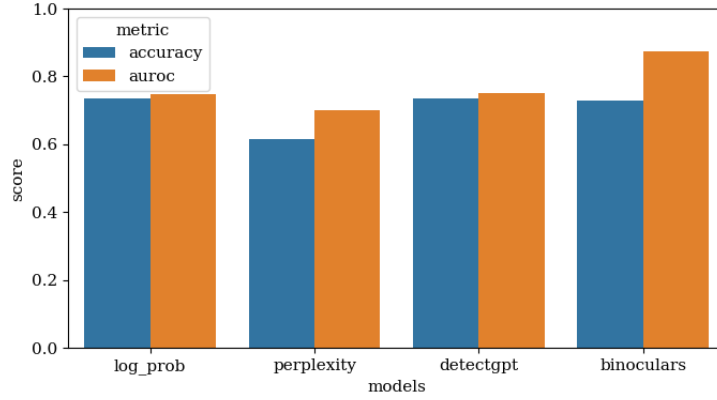


Figure 5: Detection methods comparison based on AUROC and Accuracy.

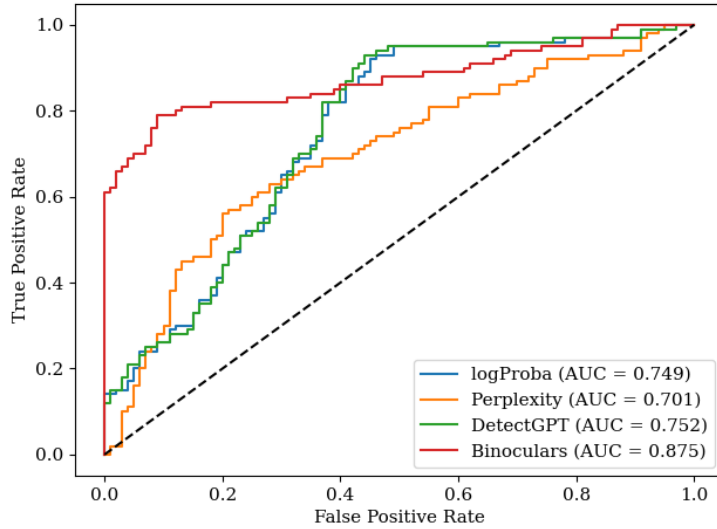


Figure 6: ROC curves for zero-shot AI-text detection methods. The Binoculars method achieves the best discrimination performance (AUC = 0.875), followed by DetectGPT (0.752), log-probability (0.749), and perplexity (0.701). The dashed line corresponds to random classification.

5 Discussion and Limitations

This work assesses zero-shot methods to detect LLMs-generated texts, particularly Binoculars and DetectGPT. Some limitations in the experimentation process must be acknowledged:

Sample size. Because of computational constraints, the experiment dataset is restricted to 200 texts. This small sample size limits the statistical reliability of the metrics. The results should be interpreted as indicative, and replication on the whole corpus of 362,868 texts is necessary to draw statistically significant conclusions.

Surrogate model mismatch. For both DetectGPT and Binoculars we use GPT-2 and GPT-2-medium as scoring models, which are significantly smaller than the models used in the original papers, Falcon-7B and Falcon-7B-Instruct for Binoculars, and GPT-NeoX for DetectGPT. This mismatch likely downgrades performance, as larger models produce more reliable log-probability estimates. The measured AUROC values should therefore be seen as lower bounds on the performance achievable with these models when properly trained.

Perturbation model quality. DetectGPT’s perturbation distribution is implemented using T5-small rather than T5-large as in the original paper. Smaller models produce lower-quality perturbations, which weaken the log-probability curvature signal and explain the modest gain of DetectGPT over the raw log-probability baseline (AUROC 0.752 vs 0.748). Also the number of perturbations should be increased as it is only 5 in our setting.

Threshold sensitivity. The classification thresholds are calibrated by maximizing F1 on the validation set. As in real-world setting the classes are more imbalanced, the F1 maximization procedure might not be optimal.

Future work should investigate threshold calibration strategies that are robust to class imbalance, such as optimizing at a fixed false positive rate and dynamic threshold adjustment.

References

- Michael A. Covington and Joe D. McFall. Cutting the Gordian knot: The moving-average type-token ratio (MATTR). *Journal of Quantitative Linguistics*, 17(2):94–100, 2010. doi: 10.1080/09296171003643098.
- Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy: Industrial-strength natural language processing in Python, 2020.
- Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. Detectgpt: Zero-shot machine-generated text detection using probability curvature. 2023. URL <https://arxiv.org/abs/2301.11305>.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. 2023. URL <https://arxiv.org/abs/1910.10683>.
- Abhimanyu Hans, Avi Schwarzschild, Valeriia Cherepanova, Hamid Kazemi, Aniruddha Saha, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Spotting llms with binoculars: Zero-shot detection of machine-generated text. 2024. URL <https://arxiv.org/abs/2401.12070>.